Capstone Assignment

TC6035 · Advanced Algorithms and Bioinspired Techniques · Part 1

Prof. Víctor Adrián Sosa Hernández

October 12, 2026

1 What this assignment is

Solve a mixed discrete–continuous optimization problem on an instance that is yours alone, under a fixed evaluation budget, using the techniques developed in class — and defend a claim about them with statistics you produced.

Released	after Session 2
Deliverable	one executed notebook
Due	**26 October 2026**
Weight	the whole of Part 1′s 25% of the course
Work	alone or in a pair, as you prefer

The algorithms are not the deliverable. What is assessed is how you compose them into a solver for a problem none of them handles alone, and what you can defend about the result.

2 The problem

Wireless sensor network placement. A 100×100 field carries a non-uniform demand field on a 50×50 grid — $2\,500$ points, each with weight $d_j > 0$. You have $n = 80$ candidate mounting sites and must decide two things at once: which sites to activate, and how much power to give each.

$$\mathbf{x} = (\mathbf{z}, \mathbf{p}), \qquad \mathbf{z} \in \{0,1\}^{80}, \qquad p_i \in [1, c_i]$$

160 decision variables: 80 binary and 80 real. c_i is site i's own power cap, and the caps differ between sites — that heterogeneity is what couples the two layers. The search space is mixed, $\Omega = \Omega_d \times \Omega_c$.

2.1 The objective

$$\max_{\mathbf{x}\in\Omega} f(\mathbf{x}) = C(\mathbf{x}) - 0.6\,E(\mathbf{x}) - 0.25\,I(\mathbf{x}) - P(\mathbf{x})$$

A sensor at site i covers a disc of radius $r_i = 4\sqrt{p_i}$. With k_j the number of active sensors covering demand point j, and $D = \sum_j d_j$:

	definition	behaviour
C	$\frac{1}{D}\sum_{j:k_j>0} d_j$	demand-weighted covered fraction

	definition	behaviour
E	$\frac{1}{15600}\ \left(\ 25\lvert\mathbf{z}\rvert + \sum_{i:z_i=1} p_i^2\ \right)$	25 per activated sensor, plus a term quadratic in power
I	$\frac{1}{D}\sum_{j:k_j>0} d_j(1-1/k_j)$	redundancy, saturating
P	0 if connected, else $2.0 + 0.1(m-1)$ for m components	infeasibility penalty

Two exponents carry the design. Coverage area grows as r^2 and r as $\sqrt{p}$, so coverage grows linearly in power while energy grows quadratically: power has an interior optimum. The fixed cost per activated sensor is what makes finding a *small* covering set the hard part.

Constraint. Active sensors must form one connected component, where i and j communicate when $d_{ij} \le 2.5\min(r_i, r_j)$. The continuous layer therefore decides whether the discrete layer is feasible at all.

Infeasibility is handled by penalty, not rejection. `evaluate()` returns a scalar that already includes it; `evaluate_verbose()` also reports `feasible` and the component count, so a repair-based approach is equally available. Both cost one evaluation.

Direction. This maximizes. Most metaheuristics literature minimizes, so a formulation copied from a textbook will have its comparisons backwards. Flip them, or optimize $-f$.

2.2 Three sensors, worked

Activate only sites 8, 40 and 45 of the reference instance and sweep their power. Only $\mathbf{z}$ and $\mathbf{p}$ are yours to set; the caps $c = (23.0, 19.7, 18.8)$, the distances $(43.5,\ 31.5,\ 32.6)$, the demand field and every weight above are fixed by the instance.

p asked	effective power	C	E	I	comps	f
8	8, 8, 8	0.2100	0.0171	0.0000	3	-2.0003
12	12, 12, 12	0.3120	0.0325	0.0000	1	$+0.2925$
16	16, 16, 16	0.4144	0.0540	0.0000	1	$+0.3820$
20	20, 19.7, 18.8	0.4904	0.0782	0.0076	1	$+0.4415$
25	23.0, 19.7, 18.8	0.5112	0.0866	0.0107	1	$+0.4566$

Effective power is what a sensor actually transmits after its request is clipped at its own cap. *comps* is the number of connected components; feasible means exactly one.

Read it row by row. Each row shows a different term of the objective taking over.

At $p = 8$, the penalty decides everything. The discs have radius $4\sqrt{8} = 11.3$, so the communication threshold is $2.5 \times 11.3 = 28.3$ — and the closest pair of these three sites is 31.5 apart. Nothing reaches anything. Three isolated sensors means three components, so the penalty is $2.0 + 0.1 \times 2 = 2.2$:

$$f = \underbrace{0.2100}_{C} - 0.6\,\underbrace{(0.0171)}_{E} - 0.25\,\underbrace{(0.0000)}_{I} - \underbrace{2.2}_{P} = -2.0003$$

The coverage is real — 21% of the demand is served — and it is irrelevant next to an infeasibility of 2.2. That is the penalty doing its job: it makes every infeasible solution worse than every feasible one.

At $p = 12$ the graph connects and f jumps by 2.3. The radius reaches 13.9 and the threshold 34.7, which now exceeds every pairwise distance. The penalty vanishes. Notice what changed: not *which* sites are active — the same three — but *how much power* they transmit. The continuous layer decided whether the discrete layer was feasible. That is the coupling, and it is why optimizing one layer with the other frozen loses.

From $p = 16$ the caps bite. Sites 40 and 45 are capped at 19.7 and 18.8, so once you ask for more than that, two of your three sensors ignore you. The row for $p = 25$ has the same effective power as $p = 22$ for those two, which is why it buys only $+0.0019$. Asking for power you cannot have costs nothing and achieves nothing.

Interference only appears at $p = 20$, once the discs are wide enough to overlap, and grows after that. Energy grows quadratically throughout. Together they mean power has an interior optimum: for this trio it is $p = 22.75$, giving $f = +0.4574$. Not the maximum you are allowed — the maximum that pays.

> **i** Note
>
> Three sensors is a reading exercise, not a solution. A tuned solution on this instance activates about 20 of the 80 sites at an average power near 11, reaching $C = 0.887$ and $f = +0.747$. Covering far more demand with far less power per sensor is the whole problem.

3 Your instance

Your instance is derived deterministically from your student ID: unique to you, regenerable on any machine, and calibrated so nobody gets a materially harder problem than a classmate.

The library `tc6035` and the starter notebooks are on Canvas as `tc6035-starter.zip`. It is not on PyPI. In Colab:

```
!unzip -o tc6035-starter.zip
import sys; sys.path.insert(0, "tc6035-starter/src")
```

Locally, Python 3.10+ with `numpy`, `scipy`, `matplotlib`, `jupyter`, and `src` on the path.

```
from tc6035 import build_instance, SensorPlacementProblem

instance = build_instance("A01234567", salt=0)   # your ID, and salt=0
problem  = SensorPlacementProblem(instance, budget=5000, seed=seed)
```

Run `notebooks/00_instance_check.ipynb` first. It builds your instance, draws it, and demonstrates the budget. If it runs, your environment is correct.

Use `salt=0` unless you have been told otherwise. The salt exists only to keep difficulty comparable: an instance whose reference score falls outside the calibrated band is regenerated with the next salt, so the mapping from ID to instance stays deterministic. Exactly one instance in this group needed it. If that one is yours you have been notified individually, and the value is on Canvas; nobody else has to do anything.

You may work alone or in a pair, whichever you prefer. **Either way the study is on one instance**: a pair picks one of their two and names whose it is in the first cell. Both members are responsible for the whole submission, and the video is what distinguishes them individually.

`instance` exposes `candidates`, `power_cap`, `grid_points`, `demand`, `dist_to_grid`, `dist_between` and `spec`.

3.1 If you would rather use your own problem

You may. Take one of two routes, declare which in the first cell, and state where each problem comes from, its decision variables, its objective and its constraints.

Route A — one mixed problem. A single problem with both layers, like the one above: a discrete choice and continuous parameters that are only meaningful together. Everything in this document applies unchanged, M3 included.

Route B — two separate problems, one discrete and one continuous. Solve the discrete one with the techniques built for choosing, the continuous one with those built for $\mathbb{R}^n$. There is no coupled hybrid to build, so M3 becomes a cross-comparison instead: run every technique on both problems, including the ones that do not fit, and account for what the mismatch costs. The claim you defend is the same one — which method belongs on which layer, and what your evidence licenses — but you argue it from the mismatch rather than from a composition.

Either route keeps the budget at 5 000 evaluations per run, the protocol in Section 6, and the rubric. Route B is more work, not less: two problems means two experimental sections. Pick a problem you can state in a paragraph and evaluate in milliseconds — if one run takes a minute, 30 seeds times six methods will not finish.

The budget is 5 000 objective evaluations per run, enforced: the problem object raises `BudgetExceeded` on the 5 001st, and a batch of n costs n, not one. Cost here is counted in evaluations, not asymptotics, and where yours went is graded.

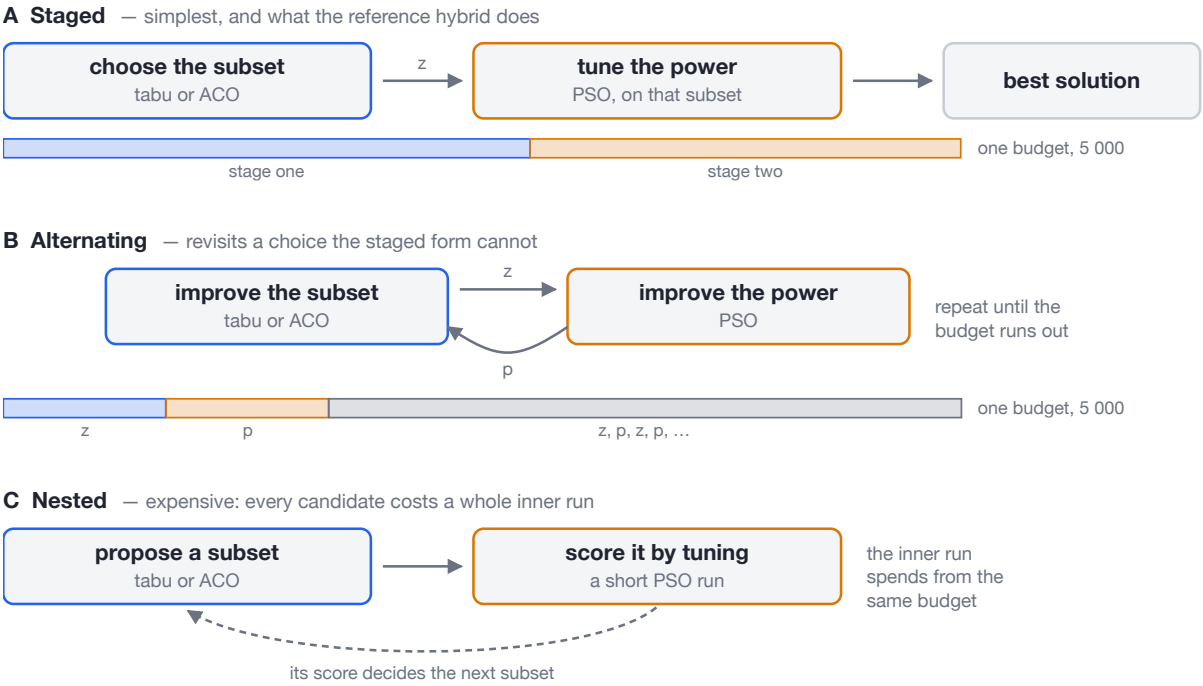
4 What you build, and what you decide

Five techniques, all developed in class. They do not all search the same thing, and that is what makes composing them necessary:

technique	moves	so on its own it
Monte Carlo	both layers	searches everything, badly — it is the baseline
simulated annealing	both layers	searches everything, blindly
tabu search	$\mathbf{z}$ only	cannot set power
ant colony optimization	$\mathbf{z}$ only	cannot set power
particle swarm optimization	$\mathbf{p}$ only	cannot choose the subset

The last three each see half the problem. Pairing one of the two subset methods with the swarm gives you a solver for the whole of it, and the point of the exercise is that the halves are coupled: the best subset depends on the power you will give it, and the power you can give depends on which sites you chose.

Composing them is the assignment. Three shapes it can take:



A is the simplest and is what the reference hybrid does. Pick the subset, then tune its power, splitting the budget between the two. Its weakness is that stage one chooses without knowing what power stage two will manage.

B revisits that. Alternating lets a subset chosen early be reconsidered once you have seen what power it can actually carry. It costs bookkeeping and a decision about how long each turn lasts.

C is the honest version of the idea and usually too expensive: scoring one candidate subset properly means running the swarm on it, so a few hundred candidates consume the budget before the outer search has gone anywhere. If you choose it, say how you kept the inner runs short enough to matter.

None of the three is the intended answer, and a fourth shape of your own is welcome. What is not negotiable is the budget: every arrow in the diagram spends from the same 5 000.

What is fixed: these five techniques, your instance, the 5 000-evaluation budget, the protocol of Section 6, and the chaining between milestones.

What is yours:

- which technique handles which layer, and why that pairing
- how they are composed — alternating, nested, staged, or something else
- the representation, the operators, the parameters, and how you tune them
- how you measure what you claim
- how you organize your notebook inside the required skeleton

There is no intended answer. A composition that fails, analysed honestly, scores above one that works and is not explained.

5 The four milestones

Each milestone is a question your notebook must answer. M1, M2 and M3 open by citing the previous one's values by name; one that cites nothing is not scored. The session named is where the material is taught.

5.1 M0 — What kind of problem is this? *(Session 1)*

State the problem formally, including the feasibility mechanism you chose and why. Characterize your instance well enough to support a claim about *why* it is hard. Show whether the two layers are coupled — that is testable, so test it under one budget. Then Monte Carlo as the baseline every later method must beat, and simulated annealing with its schedule justified by a measurement.

Carry forward: your baseline median, and any constant later work depends on.

5.2 M1 — What does memory buy? *(Session 2)*

Tabu search on the discrete layer: say what your memory structure stores and why that is the right thing to store. PSO on the continuous layer. Then compare tabu against your M0 annealing — and be careful what the comparison measures, because if they are not searching the same space that is a finding about scope.

Carry forward: the configurations you selected, and the evidence that selected them.

5.3 M2 — What is the swarm actually doing? *(Session 3)*

Examine PSO rather than only running it: how the topology changes behaviour, whether your parameters sit in a stable regime, and whether your runs stop improving before the budget ends. That last has a correct negative answer — reporting stagnation you did not observe is worse than reporting none. Then ACO as subset construction, showing what the pheromone converges to and whether your bounds bind.

Carry forward: your best PSO configuration, and the evidence.

5.4 M3 — What does your comparison license? *(Session 4)*

Compose at least two techniques into a solver for the whole problem, stating the design rationale before showing the result. Run the comparative study across everything you built. Then state precisely what your data does and does not license you to claim, and what experiment would falsify your main claim.

On route B there is nothing to compose. Instead, run every technique on both of your problems, including the ones built for the other layer, and account for what the mismatch costs. The rest of the milestone is identical: the same study, the same statement of what your data licenses, the same falsifying experiment.

Finish with the AI ledger (Section 8) and the video (Section 9).

> ⚠ Warning
>
> A composed algorithm must carve its stages out of **one** budget. A second `SensorPlacementProblem` silently doubles it. Use `problem.limited(n)`:
>
> ```
> with problem.limited(int(problem.budget * split)):
>     subset = choose_subset(problem, seed)   # stage one stops here
> tune_power(problem, seed, subset)           # spends the remainder
> ```

6 Experimental protocol

Every comparison must satisfy all of the following.

requirement	why
30 independent seeds per configuration	below this the variance of a stochastic search is not estimable
median and IQR, never best-of-N alone	best-of-N reports your luck, not your algorithm
non-parametric test — Wilcoxon for pairs, Friedman for more	metaheuristic outcomes are neither normal nor independent
Holm correction beyond one comparison	uncorrected multiple comparisons manufacture significance
paired designs, same seeds across algorithms	removes seed variance from the comparison
effect size, not only significance	at $n = 30$ a trivial difference can reach significance

State your seeds. This is the most common place where confidently-written but wrong analysis appears, including from language models: a t-test on raw run outcomes is a rejection-worthy error in this field.

7 Your own visualization

At least one figure showing an algorithm's internal state — swarm trajectory, pheromone evolution, the tabu list blocking a reversal — generated from your own logged run data. Graded on whether it exposes the mechanism, not on polish.

8 Use of AI is permitted, documented and graded

You may use language models. What is assessed is your judgment about the output, not the volume of it. Keep an AI ledger as a section of your notebook: tool and version, prompt, what it produced in your words, your verdict (accepted, modified, rejected), your justification, and the verification you performed — "it ran" is not verification.

The graded part is the error. Document at least one concrete case where the model was wrong, with the evidence that revealed it. Fifty accepted prompts and no critique scores poorly regardless of length.

If you used no AI, say so in a signed declaration. That is complete and unpenalized, scored on the independent verification it cites.

9 Video defense

Each student individually records at most 5 minutes, showing their own instance and their own results, and answering two questions drawn at recording time from the published bank. The link goes in the `# Video` section of your notebook.

In a pair, both members record separately. The video is an individual multiplier on the pair's score, capped at 1.0 — it can confirm or reduce a grade, never inflate one. No video means a zero multiplier.

10 Delivery

One notebook, one submission to `TC6035 P1 — Capstone`, due **26 October 2026**. The four milestones are the structure of the work and of the rubric, not four submissions.

File	`tc6035-A01234567.ipynb`, or both IDs if a pair
Pairs	submit once from either member; the other is named in the first cell
Size	under 20 MB. Do not submit run logs — summarize them and keep them locally
Before saving	run every cell top to bottom. A notebook whose outputs are stale is graded on its code alone, which is worth the fewest points

Required top-level sections, in this order. This is navigation, not a template — organize as you like inside each.

```
# Identification     # Formulation      # Implementations
# M0   # M1   # M2   # M3
# Analysis           # AI Ledger        # Video
```

`# Implementations` holds every method you built and an `ALGORITHMS` dict registering them by name; the notebook is the only file you submit, so anything defined elsewhere does not exist. Each milestone section ends in `## Carry forward`, and M1–M3 open by citing the previous one's values as named constants in code, not prose.

Parameter conditions and starting points are in `PARAMETERS.md` in the starter pack.

11 Rubric

$$\text{final}_i = \left(\text{ notebook out of } \ 100 \ \right) \times \text{video factor}_i$$

The notebook score is shared by a pair; the video factor is individual.

criterion	pts	top band requires
Correctness and protocol	30	30 paired seeds, median and IQR, the right test with Holm, an effect size, budget respected and shown to be. Algorithms match their description.
Analysis and interpretation	35	Falsifiable claims, tested. Mechanistic explanation referring to the landscape, the memory structure, the budget. The composition defended as a design decision. Correct bounds on what the data licenses. Negative results analysed, not hidden.

criterion	pts	top band requires
AI ledger	15	At least one concrete case where the model was wrong, the evidence that caught it, and what you did instead. Or a no-AI declaration citing independent verification.
Budget accounting	10	Evaluations accounted for per phase, the allocation defended, an alternative considered against evidence.
Visualization	10	Exposes a mechanism, drawn from your own logged data, and read in the text.

Each criterion bands down from there: **Proficient** is correct and correctly interpreted but partly descriptive; **Developing** reports what happened without explaining why; **Inadequate** describes outputs with no analysis, or draws conclusions the data does not support.

factor	the video
1.00	Explains their own contribution and can answer about their partner's. Handles the drawn questions without hesitation.
0.85	Confident and correct on their own work, vague on the rest.
0.60	Hesitates on their own results. Describes what was done but not why.
0.30	Cannot account for results attributed to them.
0.00	No video, or unable to defend any part.

Not penalized: a negative result honestly analysed; a composition that fails to beat its components, if the rationale came first; declaring no AI use; disagreeing with the course material, with support.

Automatic: a milestone citing nothing from its predecessor is not scored; more than two members is rejected; outputs inconsistent with the code are graded on code alone; fabricated results are referred under academic integrity.